Department of Computer Science and Engineering

CSA06 - Design and Analysis of Algorithms

CO1 - Assessment Tool 1: IT Application-Based Problem Solving

Scalability Audit of an E-Commerce Platform (ShopSphere)

Student Reg. No.	192465067
Student Name	N D YOGESHWAR
Course outcome assessed	Determine algorithm efficiency using asymptotic notations and mathematical techniques including the Master Theorem and the substitution method
Mode	Individual
Assumption	Servers execute 10^8 basic operations per second; nightly window of 10 minutes = 6 x 10^{10} operations

Notation. log means $\log_2$. n is the number of products in the catalogue. The two sizes studied are the present n = 100,000 and the expected n = 1,000,000.

(a) Understanding the problem

Module	**Input size**	**Basic operation and where it occurs**	**Expected growth of the count**
M1 duplicate detector	n products	Comparison Product[i].title = Product[j].title, the first test of the IF in the inner loop. It is executed for every pair (i, j) with i < j. The seller comparison is made only when the titles are equal, so it is executed no more often.	Quadratic. n x 10 gives 100 times more comparisons.
M2 rate limiter	n, the largest window that is audited	The call Audit(Request[j]) in the inner FOR loop, executed once for every request in every window.	The windows have sizes 1, 2, 4, ..., so the total is a geometric sum, linear in n.
M3 search, S1	n products in the unsorted catalogue	Comparison of the key with the product in the scan, once per examined entry.	Linear in n for a failed search; n/2 on average for a hit.
M3 search, S2	n entries in the sorted index	Comparison of the key with the middle entry, once per recursion or loop step.	Logarithmic: one more comparison each time n is doubled.
M4 recommendation engine	n, size of the cu stomer-product data	The linear-time merging of partial results, f(n) = n operations at each node of the recursion tree.	Current design: quadratic, because 4 subproblems of half size are created. Redesign: n log n.
M5 sales analytics	n sales records	The comparisons of the sort that is done at every level, f(n) = n log n at the top node.	Slightly more than n log n: all levels together cost about n $\log^2$ n.

The report generator is not a recursive module; the profile gives its operation count directly as f(n) = $3n^2$ + 10n log n + 50, in which the term $3n^2$ will dominate.

(b) Analysis of the iterative modules

Module M1

The outer loop variable i runs from 0 to n - 2. For a fixed i the inner loop variable j runs from i + 1 to n - 1, which is n - 1 - i values. The comparison is executed at every iteration, because there is no early exit (a duplicate is only marked).

```
C(n) = sum for i = 0 to n-2 of (n - 1 - i)
     = (n-1) + (n-2) + ... + 2 + 1
     = (n - 1) n / 2
     = n^2/2 - n/2
```

Time complexity: $C(n) = n(n - 1)/2$ is of the form $n^2/2 - n/2$, therefore $T(n) = \Theta(n^2)$, and the same count is obtained in the best, average and worst case, since the loops do not depend on the data. Space complexity: only the loop counters i and j are needed, so the extra space is $\Theta(1)$ (the catalogue itself, of size n, is the input). For n = 100,000 the count is 4,999,950,000 and for n = 1,000,000 it is 499,999,500,000.

Module M2

Let the values taken by i be 1, 2, 4, ... The loop runs while $i < n$. If k is the smallest integer with $2^k \geq n$, that is k = ceil(log n), then the values are $i = 2^0, 2^1, ..., 2^{k-1}$, and the outer loop therefore runs k times. In the iteration with the value i the inner loop makes exactly i calls to Audit, not n calls. The total is a geometric series:

```
A(n) = 2^0 + 2^1 + ... + 2^(k-1) = 2^k - 1,   k = ceil(log2 n)
```

Since $2^k \geq n$ and $2^k < 2n$, we get $n - 1 \leq A(n) < 2n - 1$. For n a power of two, $A(n) = n - 1$ exactly. Hence $A(n) = \Theta(n)$.

The claim of the team member is not correct as a tight bound. It is true that the outer loop runs about log n times and that the inner loop never runs more than n times, so the cost is O(n log n), and an O(n log n) bound is a valid upper bound, but it is not tight. The error is to multiply the number of iterations of the outer loop by the largest value of the inner loop: the inner loop runs $n/2^j$ times fewer in the earlier iterations, and only the last iteration costs about n/2. The tight bound is $\Theta(n)$.

Verification for n = 1,024. The values of i are 1, 2, 4, 8, 16, 32, 64, 128, 256, 512 (the value 1,024 is not less than n and the loop stops), which is k = 10 iterations. The count is $2^{10} - 1 = 1{,}023 = n - 1$. The claim of n log n would give 1,024 x 10 = 10,240 operations, about ten times more than the real count. The program output in Figure 1 shows the counted value 1,023 for n = 1,024.

(c) Recurrence of search version S2 by backward substitution

The recurrence is $T(n) = T(n/2) + 1$ with $T(1) = 1$. Assume $n = 2^k$ at first.

```
T(n) = T(n/2) + 1
     = [ T(n/4) + 1 ] + 1              = T(n/2^2) + 2
     = [ T(n/8) + 1 ] + 2              = T(n/2^3) + 3
     ...
     = T(n/2^i) + i                    (after i substitutions)
Stop when n/2^i = 1, i.e. i = log2 n:
T(n) = T(1) + log2 n = 1 + log2 n
```

Check: T(2) = T(1) + 1 = 2 = 1 + log 2; T(16) = 5 = 1 + 4. For an n that is not a power of two the halving takes the floor, and the exact closed form is T(n) = floor($\log_2$ n) + 1; for n = 1,000,000 it is 19 + 1 = 20. Time complexity: Θ(log n).

Implementation	Time	Space	Reason
Iterative (low, high loop)	Θ(log n)	Θ(1)	One comparison per halving; only low, high and mid are stored.
Recursive	Θ(log n)	Θ(log n)	The same number of comparisons, but every halving adds a stack frame, so the depth of the recursion is floor(log n) + 1.

Interpretation: with n = 1,000,000 the index needs at most 20 comparisons, and a doubling of the catalogue costs only one more. For the platform the memory of the recursive version (about 20 frames) is negligible, so both versions are acceptable; the iterative one is preferable on a busy server because it avoids the call overhead.

(d) The Master Theorem for M4 and M5

Form: T(n) = aT(n/b) + f(n), compared with $n^{\log_b a}$.

	Current design of M4	Proposed redesign of M4
Recurrence	T(n) = 4T(n/2) + n	T(n) = 2T(n/2) + n
a, b, f(n)	a = 4, b = 2, f(n) = n	a = 2, b = 2, f(n) = n
n^(log_b a)	$n^{\log_2 4} = n^2$	$n^{\log_2 2} = n$
Comparison	f(n) = n = $O(n^{2-\varepsilon})$ with ε = 1: f grows polynomially slower	f(n) = n = Θ(n): the same growth
Case	Case 1	Case 2
Result	T(n) = $\Theta(n^2)$	T(n) = Θ(n log n)

Exact solutions with T(1) = 1 are T(n) = $2n^2$ - n for the current design and T(n) = n log n + n for the redesign (both were confirmed by a program for n = 2, 16 and 1,024).

When the data doubles, the quadratic design takes T(2n)/T(n) = $(2(2n)^2 - 2n)/(2n^2 - n) \approx 4$ times as long: a catalogue twice as large means four times the work. The redesign needs T(2n)/T(n) = 2n(log n + 2)/(n(log n + 1)) = 2(1 + 1/(log n + 1)), about 2.1 times for n = 10^6, which is almost linear. The redesign is therefore not only faster but also scales proportionally.

Module M5: T(n) = 2T(n/2) + n log n

Here a = 2, b = 2, $n^{\log_2 2} = n$ and f(n) = n log n. Case 2 needs f(n) = Θ(n) and is not satisfied, since n log n is larger. Case 3 needs f(n) = $\Omega(n^{1+\varepsilon})$ for some ε > 0, but n log n / n = log n grows more slowly than any n^{ε}, so Case 3 does not apply. Case 1 fails since f is larger than n. Therefore none of the three standard cases applies; the gap between f(n) and n is only a logarithmic factor.

The extended Master Theorem covers f(n) = $\Theta(n^{\log_b a} \log^k n)$ with k ≥ 0, and gives T(n) = $\Theta(n^{\log_b a} \log^{k+1} n)$. Here k = 1, so T(n) = $\Theta(n \log^2 n)$. The same follows from the recursion tree:

```
level i has 2^i nodes, each of size n/2^i, each costing (n/2^i) log(n/2^i)
cost of level i = 2^i * (n/2^i) * log(n/2^i) = n (log n - i)
levels i = 0 .. log n - 1:
sum = n [ log n + (log n - 1) + ... + 1 ] = n log n (log n + 1) / 2
plus the n leaves of cost T(1) = 1  ->  T(n) = n log n (log n + 1)/2 + n = Theta(n log^2 n)
```

Interpretation: sorting at every level of the recursion costs a factor of log n more than sorting once. For n = 1,000,000, log n ≈ 20, so M5 does about 2.1 x 10^8 operations instead of about 2 x 10^7.

(e) Best, average and worst cases of S1 and S2

	Best case	**Average case**	**Worst case**
S1 sequential search	Key is the first product: 1 comparison	Key present, equally likely at any position: (n + 1)/2	Key is last or absent: n comparisons
S2 binary search	Key is the middle entry of the index: 1 comparison	Key present: about log n - 1 comparisons	Key absent or on the last level: floor(log n) + 1 comparisons

For n = 1,000,000: S1 needs 1, about 500,000.5 and 1,000,000 comparisons; S2 needs 1, about 18.9 and 20 comparisons. In S2 half of all the entries are on the last level of the comparison tree, which is why the average is only one or two comparisons below the worst case.

Average number of comparisons of S1 with probability p

If the product exists (probability p) and every position is equally likely, the expected number of comparisons is (1 + 2 + ... + n)/n = (n + 1)/2. If it does not exist (probability 1 - p) all n entries are examined. By the law of total expectation:

```
C_avg(n, p) = p (n + 1)/2 + (1 - p) n
            = n (1 - p/2) + p/2
n = 1,000,000 and p = 0.5:
C_avg = 0.5 x 500,000.5 + 0.5 x 1,000,000 = 250,000.25 + 500,000 = 750,000.25 comparisons
```

Interpretation: when half of the searches are for products that are not in the catalogue, the average cost rises from n/2 to 0.75n, and the failed searches contribute two thirds of the work. For S2 with p = 0.5 the average is about 0.5 x 18.9 + 0.5 x 20 ≈ 19.5 comparisons, so the presence of missing products makes almost no difference to it.

(f) Proof that $f(n) = 3n^2 + 10n \log_2 n + 50$ is $\Theta(n^2)$

Definition: $f(n) = \Theta(g(n))$ if there are positive constants c_1, c_2 and n_0 such that $c_1\, g(n) \le f(n) \le c_2\, g(n)$ for all $n \ge n_0$. Here $g(n) = n^2$.

Lower bound. For $n \ge 1$ every term of f(n) is non-negative, so $f(n) \ge 3n^2$. Hence $c_1 = 3$ works for $n \ge 1$.

Upper bound. We need $10n \log n + 50 \le (c_2 - 3)\, n^2$. Choose $c_2 = 4$, so that we need $10n \log n + 50 \le n^2$ for $n \ge n_0$. At n = 64 the left side is 10 x 64 x 6 + 50 = 3,890 and the right side is 4,096, so the inequality holds. For $n \ge 64$: $10\, n \log n = n^2\, (10 \log n / n)$, and the factor $10 \log n / n$ is decreasing for $n \ge 3$ (the derivative of $\log n / n$ is negative beyond e), so it is at most 60/64 = 0.9375. Therefore $10n \log n \le 0.9375\, n^2$, and the remaining $50 \le 0.0625\, n^2$ since 0.0625 x 64^2 = 256. So $f(n) \le 4n^2$ for $n \ge 64$.

Thus $3n^2 \le f(n) \le 4n^2$ for all $n \ge 64$, with $c_1 = 3$, $c_2 = 4$ and $n_0 = 64$, and $f(n) = \Theta(n^2)$. (A cruder but simpler choice is $c_2 = 63$ and $n_0 = 1$, since $\log n \le n$ and $1 \le n^2$ for $n \ge 1$. The values near the boundary were checked by a program: $f(59) > 4 \times 59^2$ and $f(60) \le 4 \times 60^2$, so the constant $n_0 = 64$ is safe.)

f(n) is not O(n). Suppose that $f(n) \leq cn$ for some constant c and all $n \geq n_1$. Since $f(n) \geq 3n^2$, this would give $3n^2 \leq cn$, that is $n \leq c/3$, for every $n \geq n_1$. This is false for $n = \max(n_1, \text{ceil}(c/3) + 1)$. The assumption is therefore wrong and f(n) is not O(n). In the same way f(n) is not $o(n^2)$. Interpretation: only the term $3n^2$ matters for the scalability of the report generator, and reducing the n log n or constant terms will not change its growth.

(g) Scalability decision

Module	Operation count	Operations for $n = 10^6$	Time at 10^8 op/s	Fits 10-minute window (6 x 10^{10} op)?
M1	$n(n-1)/2$	5.000e+11	5,000 s (1.4 h)	**No**
M2	$2^{20} - 1$	1.049e+06	0.01 s	Yes
S2 (one search)	floor(log n) + 1	2.000e+01	2.00e-07 s	Yes
M4 current	$2n^2 - n$	2.000e+12	20,000 s (5.6 h)	**No**
M4 redesign	n log n + n	2.093e+07	0.21 s	Yes
M5	n log n (log n + 1)/2 + n	2.096e+08	2.10 s	Yes
Report generator	$3n^2 + 10n \log n + 50$	3.000e+12	30,002 s (8.3 h)	**No**

For comparison, at the present n = 100,000: M1 needs 5.0×10^9 operations (50 s), M4 current 2.0×10^{10} (200 s) and the report generator 3.0×10^{10} (300 s); all fit in 600 s today, and all three fail after the expansion because they are quadratic, so a tenfold growth costs a hundredfold time.

Modules that fail the 10-minute window at n = 1,000,000: M1 (about 5,000 s, 83 minutes), the current M4 (about 20,000 s, 5.6 hours) and the report generator (about 30,000 s, 8.3 hours). M2, S2, the redesigned M4 and M5 are comfortably within the window (0.01 s, 0.2 microseconds, 0.21 s and 2.1 s).

Recommendations

M1 must be redesigned. The duplicate test depends only on equality of the pair (title, seller), so comparing every pair is unnecessary. Build a hash table keyed on (title, seller) and mark a product as a duplicate when its key is already in the table: expected $\Theta(n)$ time and $\Theta(n)$ space, about 10^6 operations (0.01 s). If memory is a concern, sort the products by (title, seller) in $\Theta(n \log n)$ (about 2×10^7 operations, 0.2 s) and compare only neighbouring records. Both bring the module far below the window; the price is the extra memory of the hash table or the sort.

M4 must adopt the proposed redesign. Removing the replication changes a = 4 to a = 2, which moves the recurrence from Case 1 ($\Theta(n^2)$) to Case 2 ($\Theta(n \log n)$): about 2.1×10^7 operations instead of 2×10^{12}, an improvement of about 100,000 times. The replicated partitions presumably give availability, so that requirement has to be met by another means (copying data without recomputing on it).

The report generator must also be redesigned. Its dominant term is $3n^2$, which suggests a nested loop over the catalogue; as for M1 the inner pass should be replaced by a sort followed by one scan, or by a hash-based aggregation, to reach $\Theta(n \log n)$ or $\Theta(n)$.

M5 is acceptable at about 2 seconds, but it re-sorts at every level. If the sort were done once at the top, or if a merge were used to combine already sorted halves, the recurrence would become T(n) =

2T(n/2) + n, that is Θ(n log n), saving a further factor of about 10. This is optional.

M2 and S2 need no change. M2 is linear (about one million audits), and S2 needs 20 comparisons, so it should replace the sequential search S1 for every interactive lookup, whose worst case of 10^6 comparisons (0.01 s) would cost thousands of seconds of processor time if many users search at once. The sorted index only has to be maintained when products are added.

Figure 1 shows the output of the program that checks the formulae of this report.

C:\CSA06\src> python verify.py

```
======================================================================
Date/Time: 12-10-2026 22:54:29
CSA06 CO1 - verification of the derived formulae
======================================================================
M1 comparisons vs n(n-1)/2
  n =    10  counted       45  formula       45
  n =   100  counted     4950  formula     4950
  n =  1000  counted   499500  formula   499500
M2 audits vs 2^ceil(log2 n) - 1
  n =     8  counted      7  formula      7  n-1 =     7  n log2 n =       24
  n =   100  counted    127  formula    127  n-1 =    99  n log2 n =      664
  n =  1000  counted   1023  formula   1023  n-1 =   999  n log2 n =     9966
  n =  1024  counted   1023  formula   1023  n-1 =  1023  n log2 n =    10240
  n =  1025  counted   2047  formula   2047  n-1 =  1024  n log2 n =    10251
S2 recurrence vs floor(log2 n) + 1
  n =        1  T(n) =  1  formula  1
  n =        2  T(n) =  2  formula  2
  n =       16  T(n) =  5  formula  5
  n =     1024  T(n) = 11  formula 11
  n =  1048576  T(n) = 21  formula 21
M4 recurrences (n a power of two)
  n =     2  current         6 (2n^2 - n =         6)  redesign       4 (n log2 n + n =       4)
  n =    16  current       496 (2n^2 - n =       496)  redesign      80 (n log2 n + n =      80)
  n =  1024  current   2096128 (2n^2 - n =   2096128)  redesign   11264 (n log2 n + n =   11264)
M5 recurrence vs n log2 n (log2 n + 1)/2 + n
  n =     2  T(n) =        4.0  formula        4.0
  n =    16  T(n) =      176.0  formula      176.0
  n =  1024  T(n) =    57344.0  formula    57344.0
f(n) = 3n^2 + 10 n log2 n + 50 against 3n^2 and 4n^2
  n =        1  f =            53.0  3n^2 =               3  4n^2 =               4  f <= 4n^2: False
  n =       16  f =          1458.0  3n^2 =             768  4n^2 =            1024  f <= 4n^2: False
  n =       59  f =         13963.8  3n^2 =           10443  4n^2 =           13924  f <= 4n^2: False
  n =       60  f =         14394.1  3n^2 =           10800  4n^2 =           14400  f <= 4n^2: True
  n =       64  f =         16178.0  3n^2 =           12288  4n^2 =           16384  f <= 4n^2: True
  n =     1000  f =       3099707.8  3n^2 =         3000000  4n^2 =         4000000  f <= 4n^2: True
  n =  1000000  f =  3000199315735.7  3n^2 =   3000000000000  4n^2 =   4000000000000  f <= 4n^2: True
Estimates for n = 1,000,000 at 10^8 operations per second
  M1                              5.0000e+11 ops       4999.9950 s  FAILS 600 s window
  M2                              1.0486e+06 ops          0.0105 s  ok
  S2 (one search)                 2.0000e+01 ops          0.0000 s  ok
  S1 (one search, worst)          1.0000e+06 ops          0.0100 s  ok
  M4 current                      2.0000e+12 ops      19999.9900 s  FAILS 600 s window
  M4 redesign                     2.0932e+07 ops          0.2093 s  ok
  M5                              2.0960e+08 ops          2.0960 s  ok
  report generator f(n)           3.0002e+12 ops      30001.9932 s  FAILS 600 s window
```

N D YOGESHWAR | Reg No 192465067

Figure 1: Verification of the derived formulae by counting

Code of Conduct

I N D YOGESHWAR / 192465067 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: _____ Date: _____